

Classical and Deep Learning Approaches for White Blood Cell Classification from Peripheral Blood Smear Microscopy

IMA205 Challenge — Full Technical Report

Théophile Nadiedjoa
Télécom Paris

April 2026

Contents

Abstract	2	5.7 Evaluation Protocol	7
1 Introduction	2	6 Iterative Development: The Full Journey	8
2 Related Work	2	6.1 deep_1: EfficientNet-B3 Baseline	8
3 Dataset	3	6.2 deep_2: CBAM Attention + Nucleus Crop- ping	8
3.1 Overview	3	6.3 deep_3: Late Fusion + Wiener/Laplacian + 4th Channel	8
3.2 Class Distribution and Imbalance	3	6.4 deep_4: Literature Segmentation Pipeline (Failed Experiment)	9
3.3 Visual Challenges	3	6.5 deep_5: The ConvNeXt Pivot + Data Leak- age Discovery	9
4 Part I: Classical Machine Learning	3	6.6 deep_6: Leakage Fix — True Performance	9
4.1 Cell Segmentation	3	6.7 deep_7: ResNet50 + Spatial Attention + Late Fusion	9
4.1.1 Reinhard Color Normalization	4	6.8 deep_8: ResNeXt101 + CBAM + Multi-Task	10
4.1.2 Marker-Controlled Watershed	4	6.9 deep_9: ConvNeXt-Base + SE Attention (with-morph vs. no-morph)	10
4.1.3 Nucleus Refinement via Geodesic Active Contours	4	7 Results and Discussion	10
4.2 Feature Extraction	4	7.1 Approach Comparison	10
4.3 Classification	4	7.2 Per-Class Analysis (deployed model, deep_9 no-morph)	11
4.4 Classical ML Results	4	7.3 Confusion Analysis	11
4.5 Feature Importance Analysis (LightGBM)	5	7.4 Qualitative Error Analysis	11
5 Part II: Deep Learning	5	7.5 Why Deep Learning Outperforms Classical ML	12
5.1 Deployed Pipeline Overview (deep_9 no-morph)	5	7.6 Lessons from the Iterative Journey	12
5.2 Preprocessing: HSV-Based Cell Cropping and CLAHE	6	7.7 Computational Cost and Clinical Feasibility	12
5.3 Handling Class Imbalance	6	8 Conclusion and Future Work	12
5.4 Data Augmentation	6		
5.5 Architecture: ConvNeXt-Base + SE Gate + MLP Head	6		
5.5.1 Transfer Learning with ConvNeXt	6		
5.5.2 Channel-Attention Gate and Classifi- cation Head	7		
5.6 Training Strategy	7		
5.6.1 Two-Phase Training	7		
5.6.2 Final Hyperparameters (deep_9 no-morph)	7		
5.6.3 CutMix and Mixup	7		

Abstract

White blood cell (WBC) classification from peripheral blood smear microscopy is a fundamental task in clinical hematology, yet remains challenging due to extreme class imbalance (ratio 1183:1 in our dataset), staining variability, and morphological similarity between cell subtypes. In this work, we systematically explore and compare two complementary paradigms on the IMA205 challenge (28,901 training images, 9,634 test images, 13 cell types).

Part I presents a classical machine learning pipeline following the textbook recipe recommended by Al-Dulaimi et al. [1]: Reinhard color normalization in LAB space, marker-controlled Watershed cell segmentation, Morphological Geodesic Active Contours (GAC) for nucleus refinement, then a ~ 210 -dimensional handcrafted feature vector (shape, color, texture via GLCM/LBP/HOG, Fourier descriptors), classified using SMOTE-balanced LightGBM, Random Forest, ExtraTrees, SVM-RBF, and Voting/Stacking ensembles under 5-fold stratified cross-validation, reaching macro-F1 = 0.509 (LightGBM, best of the six).

Part II traces nine successive deep-learning iterations (deep_1–deep_9), each motivated by specific failures of the previous one. Starting from a vanilla EfficientNet-B3 baseline (single 85/15 validation split, macro-F1 = 0.552), we explored CBAM attention, late fusion with handcrafted morphometry, Wiener/Laplacian preprocessing, paper-based segmentation pipelines, before pivoting to ConvNeXt-Base with Focal Loss, CutMix/Mixup, and 2-phase fine-tuning. We document a critical data leakage pitfall (deep_5: apparent F1=0.82, corrected to true F1=0.70 in deep_6), and present post-hoc experiments with ResNet50 (deep_7), ResNeXt101 with multi-task learning (deep_8), and a controlled ablation of handcrafted features (deep_9). The deployed model is deep_9 no-morph (no handcrafted features): deep_6’s architecture plus a lightweight SE-style attention gate and HSV-V CLAHE preprocessing, achieving macro-F1 = 0.7052 / accuracy = 0.937 – the best validated result across all 9 iterations.

Beyond raw scores, this work establishes that (1) learned representations dominate handcrafted features (+0.19 F1, deep_6 vs. classical ML), (2) the backbone family (ConvNeXt) mattered more than added modules: a similarly sized ResNeXt101 with CBAM and multi-task heads (deep_8, 0.665) did not reach the plain ConvNeXt + MLP (0.699), and attention/fusion gave at most marginal gains, and (3) the most impactful engineering decisions are at the *data* level (offline cropping, leakage-safe CV, rare-class-aware augmentation).

1 Introduction

The differential white blood cell count is one of the most commonly ordered laboratory tests in clinical practice. It provides critical diagnostic information for conditions ranging from infections and allergies to leukemia and other hematological malignancies [1]. Traditionally performed manu-

ally by trained hematologists examining Giemsa- or Wright-stained blood smears under a microscope, this process is time-consuming, subjective, and prone to inter-observer variability.

Automated WBC classification using computer vision has been an active area of research, with approaches spanning from classical image processing and handcrafted feature extraction [2, 3] to modern deep learning methods [4, 5]. As highlighted by Al-Dulaimi et al. [1] in their comprehensive survey, several critical challenges persist:

- **Staining variability:** Different protocols (Giemsa, Wright, May-Grünwald) lead to significant color variations.
- **Illumination differences:** Microscope lighting conditions vary between acquisitions.
- **Arbitrary cell orientation:** WBCs appear at any rotation angle on the slide.
- **Scale and position variations:** The nucleus and cytoplasm occupy varying proportions of the field of view.
- **Morphological similarity:** Several subtypes, particularly within the granulocyte lineage, share similar nuclear shapes.

In this work, we address these challenges through two complementary strategies. We first implement a classical ML pipeline to understand the discriminative value of explicit morphological features, then develop a deep learning pipeline through extensive iterative experimentation (9 iterations). This dual approach allows us to compare the strengths and limitations of each paradigm and document the lessons learned along the way.

2 Related Work

The literature on WBC classification can be broadly organized into two families: **classical image analysis pipelines** and **representation-learning approaches**.

Classical pipelines. Early and still influential work relies on explicit segmentation and handcrafted descriptors. Typical pipelines isolate the nucleus and cytoplasm using thresholding, region growing, Watershed, K-Means, Active Contours, or morphological filtering, then extract shape/color/texture features (e.g., Hu moments, GLCM, LBP, Fourier descriptors) for conventional classifiers [2, 3, 6]. Rustam et al. [7] combine texture and RGB features with oversampling of the minority classes. The review by Al-Dulaimi et al. [1] emphasizes that these approaches are interpretable and medically grounded, but strongly depend on segmentation quality and often degrade under staining/illumination variability.

Deep learning and attention mechanisms. More recent studies replace handcrafted features with end-to-end CNN feature learning [4, 5], usually via transfer learning to handle limited medical data; Toğaçar et al. [8] instead extract deep features from pretrained CNNs and feed a selected subset to conventional classifiers, a hybrid we also evaluate (CNN+SVM/RF). Attention modules (e.g., CBAM, SE) have been proposed to improve focus on discriminative regions [9, 10]. In practice, gains are task-dependent and often smaller than expected when the baseline backbone is already strong.

Positioning of this work. Our report follows this evolution explicitly: Part I implements a robust classical pipeline inspired by the survey recommendations [1], while Part II evaluates modern transfer-learning pipelines (EfficientNet/ConvNeXt/ResNet/ResNeXt), attention variants, and hybrid fusion with handcrafted features. This design enables a controlled comparison between interpretability-oriented classical vision and high-capacity learned representations.

3 Dataset

3.1 Overview

The challenge dataset consists of 28,901 training images and 9,634 test images of individual white blood cells captured from peripheral blood smear microscopy. Images are RGB PNGs of roughly 370×370 pixels (small size variations, e.g. $368\text{--}370 \times 368$). Each image shows a single WBC against a background of red blood cells and platelets, labeled into 13 cell types ranging from mature granulocytes (segmented neutrophils, eosinophils, basophils) to immature precursors (blasts, promyelocytes, myelocytes, metamyelocytes, band neutrophils), agranulocytes (lymphocytes, monocytes), reactive forms (variant lymphocytes), and rare entities (plasma cells, platelet clumps). The metadata files contain only ID and label, so the classifier must rely exclusively on the visual content.

3.2 Class Distribution and Imbalance

The dataset exhibits **extreme class imbalance**: the ratio between the largest class (SNE, 13,015) and the smallest (PLY, 11) is 1,183:1. This reflects the natural WBC distribution in peripheral blood but poses a severe challenge for classification, as a naive model predicting only SNE achieves 45% accuracy while completely failing on clinically important rare classes.

3.3 Visual Challenges

Figure 2 illustrates the morphological diversity. The staining intensity varies considerably across images. Nuclear morphology—the primary discriminative feature [2]—varies in shape, size, and texture across the 13 classes, but several

Table 1: Class distribution in the training set.

Abbrev.	Cell Type	Count	%
SNE	Segmented Neutrophil	13,015	45.0
LY	Lymphocyte	8,101	28.0
MO	Monocyte	2,746	9.5
BL	Blast	2,012	7.0
EO	Eosinophil	861	3.0
MY	Myelocyte	441	1.5
BA	Basophil	415	1.4
BNE	Band Neutrophil	391	1.4
VL	Variant Lymphocyte	366	1.3
MMY	Metamyelocyte	360	1.2
PMY	Promyelocyte	114	0.4
PC	Plasma Cell	68	0.2
PLY	Platelet clump	11	0.04
Total		28,901	

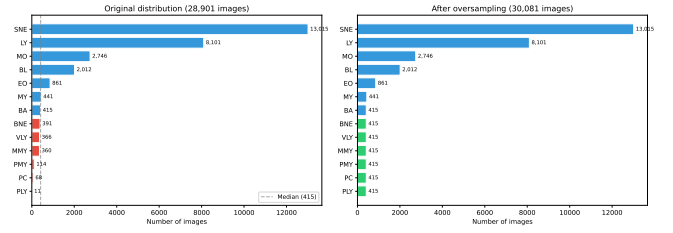


Figure 1: Original vs. oversampled class distribution. Rare classes (below median count of 415) are augmented to the median level.

classes share highly similar appearances (e.g., myelocyte vs. metamyelocyte).

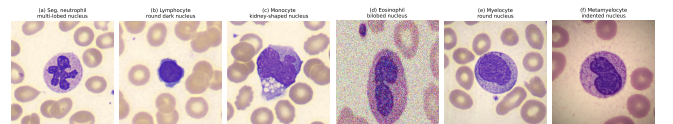


Figure 2: Representative WBC types: (a) Seg. Neutrophil, (b) Lymphocyte, (c) Monocyte, (d) Eosinophil, (e) Myelocyte, (f) Metamyelocyte. Note the similarity between (e) and (f).

4 Part I: Classical Machine Learning

The classical approach follows the traditional pipeline recommended by Al-Dulaimi et al. [1]: segmentation, feature extraction, then classification with conventional ML algorithms.

4.1 Cell Segmentation

Accurate separation of nucleus and cytoplasm is the critical first step, since nuclear morphology carries the strongest discriminative information. We implemented a three-stage segmentation pipeline:

4.1.1 Reinhard Color Normalization

Staining variability across slides causes significant color shifts that can confuse both segmentation algorithms and classifiers. We apply Reinhard color normalization [11] in LAB color space: for each image, channel-wise mean and standard deviation are matched to a reference computed from a stratified sample of 40 images per class. This standardizes the color distribution while preserving morphological structure.

4.1.2 Marker-Controlled Watershed

Cell boundary detection uses marker-controlled Watershed on the saturation channel of the HSV representation. We compute the distance transform of an initial Otsu-thresholded mask, identify foreground markers at $0.4 \times d_{\max}$ of the distance transform, and apply Watershed with these markers. The largest non-background component is selected as the cell mask.

4.1.3 Nucleus Refinement via Geodesic Active Contours

The nucleus is initially segmented via Otsu thresholding on the L channel (LAB) within the cell mask. This initial estimate is refined using Morphological Geodesic Active Contours (GAC) with 40 iterations, smoothing = 1, and no balloon pressure (edge-driven only). The inverse Gaussian gradient of the L channel serves as the edge map ($\alpha = 100$, $\sigma = 3.0$). The cytoplasm is defined as $\text{cell} \setminus \text{nucleus}$.

4.2 Feature Extraction

From the segmented cell, nucleus, and cytoplasm masks, we extract a comprehensive feature vector (210 raw features) inspired by the morphological feature sets of the literature [1, 3]:

- **Shape (A, B, D):** Area, perimeter, circularity, eccentricity, solidity, extent, convexity, axis ratio, equivalent diameter, number of lobes, 7 Hu moments — computed for both nucleus and cell (~ 25 features).
- **Color (C):** Mean, std, skewness, kurtosis of each channel in RGB, HSV, and LAB color spaces, computed separately for nucleus and cytoplasm. Plus nucleus-specific ratios (R/G, R/B, G/B) (~ 75 features).
- **Texture (GLCM + LBP):** Gray-Level Co-occurrence Matrix properties (contrast, dissimilarity, homogeneity, energy, correlation) at distances 1 and 3, angles $0, \pi/4, \pi/2$. Local Binary Patterns (8 neighbors, radius 1, uniform) with 10-bin histogram. Computed for nucleus and cytoplasm (~ 30 features).
- **HOG:** Histogram of Oriented Gradients (8 orientations, 32×32 pixels/cell, 2×2 cells/block) on 64×64 cropped regions of nucleus and cytoplasm (~ 64 features).

- **Fourier Descriptors:** 10 normalized magnitudes of the FFT of the nucleus contour, providing rotation- and scale-invariant shape encoding.

After extraction, we remove near-zero variance features (none removed: 210 remain) and features with correlation > 0.95 (131 remain), then add 20 PCA components (78.7% explained variance). The final feature set contains 151 features.

4.3 Classification

Features are standardized (zero mean, unit variance) and class imbalance is handled via SMOTE [12] ($k=3$ neighbors) at each cross-validation fold. We evaluate 6 classifiers with 5-fold stratified cross-validation. **Computational note:** the literal SMOTE call equalizes every class up to the majority class count (SNE, 13,015), which would inflate each training fold to $\sim 135k$ rows; at that scale, benchmarking showed the SVM-based models (SVM-RBF, Voting, Stacking) would require multiple hours to days to fit on this hardware (4 CPU cores). We therefore cap SMOTE oversampling at 2,000 samples/class (SMOTE_CAP in `src/classical_pipeline.py`) instead of equalizing to the majority class — documented here as a deliberate adaptation for feasibility, not a change to any other part of the pipeline:

- **LightGBM** [13] (tuned via Optuna [14]: 1755 estimators, $\text{lr}=0.034$, 54 leaves, depth 6)
- **Random Forest** (800 trees, balanced class weights)
- **ExtraTrees** (800 trees, balanced)
- **SVM-RBF** ($C=10$, balanced)
- **Soft Voting Ensemble** (LightGBM + RF + ET + SVM)
- **Stacking Ensemble** (same base learners, LogisticRegression meta-learner)

4.4 Classical ML Results

The classical ML pipeline achieves modest performance on this 13-class problem. Table 2 reports the 5-fold CV macro-F1 for all six classifiers (`notebooks/02_classical_ml.ipynb`, `results/classical_ml/cv_results.json`). LightGBM is the best single model; the ensembles do not improve over it under the capped-SMOTE setting.

OOF accuracy for the best model (LightGBM) is 0.832. The main limitations are:

1. **Segmentation failures:** The Watershed+GAC pipeline fails on a non-negligible fraction of images (visual inspection: pale staining, overlapping cells, unusual backgrounds), propagating errors to all downstream features.

Table 2: Classical ML: 5-fold CV macro-F1 per classifier.

Model	Macro-F1
LightGBM (tuned)	0.509 ± 0.024
Soft Voting	0.496 ± 0.014
Random Forest	0.477 ± 0.018
Stacking	0.459 ± 0.018
SVM-RBF	0.444 ± 0.008
ExtraTrees	0.434 ± 0.014

- Feature expressiveness:** Handcrafted features cannot capture the full complexity of chromatin patterns, granule distributions, and subtle nuclear shape variations that distinguish 13 classes. Hu moments and Fourier descriptors provide rotation invariance but lose fine-grained texture.
- Feature-label gap:** Features like GLCM or LBP describe texture statistically but lack the hierarchical abstraction that deep networks learn.

These limitations motivate the transition to deep learning in Part II.

4.5 Feature Importance Analysis (LightGBM)

To improve interpretability, we extracted global feature importances (split counts) from the LightGBM model trained on the full 151-feature set; the 30 top-ranked features are saved in `results/classical_ml/feature_importance_lightgbm.csv`. Figure 3 shows the top-ranked ones, and Table 3 lists the top 10.

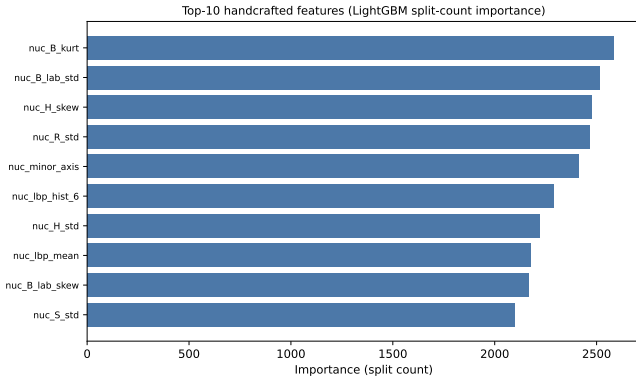


Figure 3: Top feature importances (LightGBM split counts) in Part I.

Grouping the 30 top-ranked features by family, nucleus color statistics dominate (15 features, $\approx 52\%$ of the top-30 split count), followed by LBP texture (5 features, $\approx 18\%$), HOG (3, $\approx 8\%$), shape (minor axis and area, $\approx 7\%$), two PCA components ($\approx 6\%$), and a single GLCM (dissimilarity), Fourier and Hu-moment feature each ($\approx 3\%$ each). All non-PCA features in the top 30 describe the nucleus; no cytoplasm

Table 3: Top-10 LightGBM features in Part I (split-count importance). All are nucleus features.

#	Feature	Family	Splits
1	nuc_B_kurt	Color (RGB)	2586
2	nuc_B_lab_std	Color (LAB)	2513
3	nuc_H_skew	Color (HSV)	2476
4	nuc_R_std	Color (RGB)	2466
5	nuc_minor_axis	Shape	2414
6	nuc_lbp_hist_6	Texture (LBP)	2289
7	nuc_H_std	Color (HSV)	2221
8	nuc_lbp_mean	Texture (LBP)	2174
9	nuc_B_lab_skew	Color (LAB)	2166
10	nuc_S_std	Color (HSV)	2098

feature appears. Notably, the classical morphometric descriptors often emphasized in the literature (circularity, eccentricity, nucleus-to-cell ratio) are *not* in the top 30: the model relies mostly on higher-order color/intensity statistics and micro-texture of the nucleus, i.e. chromatin appearance, rather than on coarse geometry. Split-count importance is only a rough proxy (it favours continuous, frequently split features), so this ranking should be read qualitatively.

5 Part II: Deep Learning

The deep learning approach bypasses explicit segmentation and feature engineering, instead learning discriminative representations end-to-end. Our work involved **9 iterative experiments** (`deep_1`–`deep_9`), each addressing specific failure modes identified in the previous iteration. We first describe the deployed pipeline (`deep_9 no-morph`, i.e. without handcrafted features), then detail the complete iterative journey, including `deep_6` — the leakage-fix milestone that established the architecture `deep_9` refines.

5.1 Deployed Pipeline Overview (`deep_9 no-morph`)

The deployed model, illustrated in Figure 4, consists of six stages: (1) cell-focused preprocessing via offline HSV-score cropping (done once), (2) on-the-fly CLAHE on the V channel of HSV (clip limit 2.0, 8×8 tiles), applied to every image at train, validation and test time before augmentation, (3) geometric and photometric augmentation (training only), (4) a ConvNeXt-Base backbone, (5) a lightweight SE-style channel-attention gate on the pooled features, and (6) an MLP head, evaluated leakage-free. It is `deep_6`'s architecture (§6.6) plus this attention gate and the CLAHE step; the handcrafted-morphology fusion branch that the `deep_9` code supports is disabled here (the `no-morph` variant, see §6.9), since the ablation showed it makes no meaningful difference for this backbone.

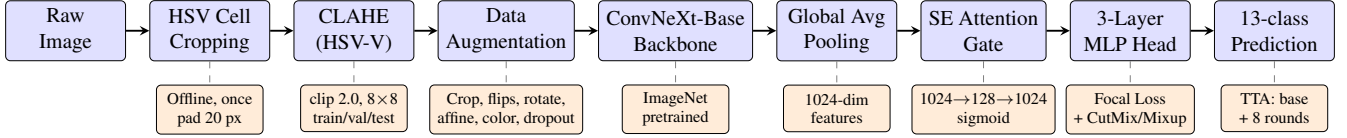


Figure 4: Deep learning pipeline (deep_9 no-morph). Offline HSV cropping removes background distractors, CLAHE on the HSV V channel equalizes local contrast (at train, validation and test time), augmentation builds geometric and photometric robustness (training only), ConvNeXt-Base learns hierarchical features, an SE-style gate reweights channels, and Focal Loss handles class imbalance. The handcrafted-morphology fusion branch this architecture supports is present in code but disabled in this no-morph variant.

5.2 Preprocessing: HSV-Based Cell Cropping and CLAHE

Unlike the classical approach which requires precise nucleus/cytoplasm segmentation, the DL approach only needs a coarse cell crop to remove background clutter. We designed an HSV-based algorithm that scores each pixel:

$$S(x, y) = 0.55 \cdot H_s + 0.25 \cdot S_s + 0.20 \cdot V_s$$

where H_s measures proximity to hue ≈ 140 (purple), S_s measures saturation, and V_s penalizes bright pixels. After thresholding at the 92nd percentile, morphological cleanup, and selection of the largest connected component near the image center, the mask is dilated with a 45×45 elliptical kernel and the bounding box (with 20px padding) defines the crop.

This preprocessing is performed **once, offline**, and saved to disk, which removes the runtime cropping bottleneck of the earlier iterations; crop quality was checked visually (deep_6 QC, §6.6). With offline crops, the deployed model trains at ≈ 84 s/epoch (frozen backbone) and ≈ 140 s/epoch (fine-tuning) on a single RTX 3090.

On top of the stored crops, deep_9 applies CLAHE (contrast-limited adaptive histogram equalization) to the V channel of the HSV representation (clip limit 2.0, 8×8 tiles) on the fly. This step is deterministic and applied to every image at train, validation and test time, *before* augmentation; hue and saturation are left unchanged.

5.3 Handling Class Imbalance

We combine three strategies:

1. **Offline oversampling:** The 6 rarest classes (below median count 415) are augmented to 415 images each via random geometric transforms.
2. **WeightedRandomSampler:** Sampling weights proportional to $n_c^{-1/2}$ (inverse square root of the class count) balance batch composition during training.
3. **Focal Loss** [15]: $\mathcal{L}_{FL} = -(1 - p_t)^\gamma \log(p_t)$ with $\gamma = 2.0$, combined with label smoothing [16] ($\epsilon = 0.05$), down-weighting easy/common examples.

Data leakage prevention: Augmented copies must stay in the same split as their parent. An earlier iteration (deep_5)

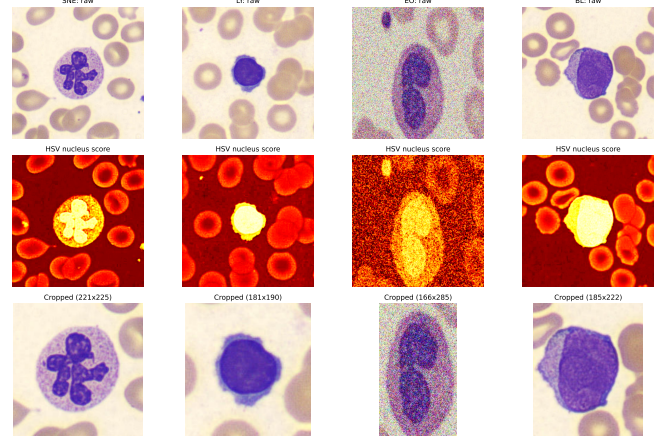


Figure 5: HSV-based cell cropping. Top: raw images. Middle: HSV score heatmaps. Bottom: cropped outputs.

that split *after* augmentation inflated F1 from 0.70 to 0.82. We fixed this in deep_6 by splitting only original images, then assigning augmented copies to their parent’s split (see Section 6.6).

5.4 Data Augmentation

The training-time augmentation of the deployed model (deep_9 no-morph) uses Albumentations [17] with geometric transforms and moderate photometric perturbation (Table 4); validation and test images are only resized to 224×224 and normalized.

Note: Apart from the small hue jitter in ColorJitter, this pipeline does *not* include explicit stain-specific augmentation (HSV shifts, RGBShift); these were used in deep_2. CLAHE is not an augmentation here but a deterministic pre-processing step shared by training and inference.

5.5 Architecture: ConvNeXt-Base + SE Gate + MLP Head

5.5.1 Transfer Learning with ConvNeXt

Al-Dulaimi et al. [1] warn that training deep networks from scratch requires datasets far larger than typical medical imaging datasets. Following this guidance, we use transfer learning with ConvNeXt-Base [18] (torchvision, pre-trained on

Table 4: Training augmentation of `deep_9 no-morph` (applied after CLAHE, in this order).

Transform	Prob.	Challenge Addressed
RandomResizedCrop(224, scale 0.75–1.0, ratio 0.9–1.1)	0.7	Scale/position variation
Horizontal flip, vertical flip, RandomRotate90	0.5 each	Arbitrary orientation
Rotate($\pm 180^\circ$)	0.9	Full 360° coverage
Affine (translate $\pm 10\%$, scale 0.85–1.15, shear $\pm 8^\circ$)	0.5	Geometric robustness
ColorJitter (B 0.3, C 0.3, S 0.2, H 0.03)	0.7	Brightness/contrast/stain var.
ToGray	0.2	Reliance on shape over color
One of GaussianBlur / GaussNoise	0.2	Acquisition artifacts
Resize(224×224) + ImageNet normalize	1.0	Uniform input
CoarseDropout (1–3 holes, 16–32 px)	0.25	Occlusion robustness

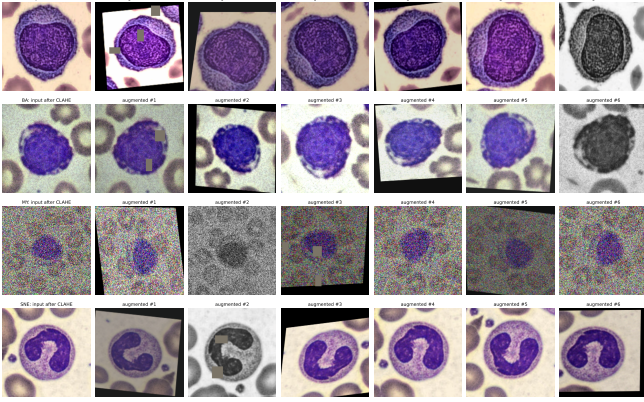


Figure 6: Training-time augmentation of the deployed model (`deep_9 no-morph`): CLAHE-processed crop (left) and augmented views drawn from the actual training pipeline (CLAHE followed by the transforms of Table 4).

ImageNet; 87.6M backbone parameters), which modernizes the CNN architecture with design elements from Vision Transformers while retaining CNN efficiency. Our architecture choice evolved from EfficientNet-B3 (12M params, single-split F1 = 0.552) to ConvNeXt-Base (val F1 = 0.70+). The comparison with the similarly sized ResNeXt101 (`deep_8`, §6.8) indicates that the backbone family, not only its size, drove this gain.

5.5.2 Channel-Attention Gate and Classification Head

After global average pooling, the 1024-dimensional feature vector passes through a lightweight SE-style [10] channel-attention gate ($1024 \rightarrow 128 \rightarrow 1024$, sigmoid, 263,296 params) that reweights features before the head, then a 3-layer MLP: $1024 \rightarrow 512 \rightarrow 256 \rightarrow 13$ with ReLU and dropout ($p = 0.3$; 659,469 params). The full model has 88,487,181 parameters (backbone 87,564,416). Heavier attention mechanisms (CBAM, spatial attention) and handcrafted feature

branches were also tried (Sections 6.7–6.9) and provided negligible-to-no gains; the `deep_9` step (SE gate + CLAHE) is the one change that stuck, giving a small edge (`deep_6` 0.6994 $\rightarrow$ `deep_9 no-morph` 0.7052) at negligible parameter cost. The architecture otherwise remains deliberately simple—no CBAM, no spatial attention, no multi-task heads.

5.6 Training Strategy

5.6.1 Two-Phase Training

- Phase 1 – Warmup** (epochs 1–5): Backbone *frozen*; only the SE gate and MLP head are trained (922,765 trainable params: gate 263K + head 659K), AdamW [19], $\text{lr}=3 \times 10^{-4}$, cosine annealing with warm restarts.
- Phase 2 – Fine-tuning** (up to epoch 60): Backbone *unfrozen*; AdamW with the *same* learning rate 10^{-4} for backbone and head (no differential learning rate), weight decay 10^{-4} .

Early stopping (patience=10 on val macro-F1) prevents overfitting. Training uses batch size 16 with 4-step gradient accumulation (effective=64), AMP fp16 mixed precision, and gradient clipping at norm 1.0. The best validation macro-F1 (0.7052) was reached at epoch 21, and training stopped early at epoch 31.

5.6.2 Final Hyperparameters (`deep_9 no-morph`)

For reproducibility, Table 5 consolidates the training configuration used for the deployed model. The figure-generation scripts and the notebooks expect the Kaggle challenge data to be placed in `data/`; the data itself is not versioned in the repository.

5.6.3 CutMix and Mixup

CutMix [20] ($\alpha = 1.0$) and Mixup [21] ($\alpha = 0.2$) are applied as batch-level augmentations, CutMix being chosen with probability 0.5 and Mixup otherwise. **Crucially**, both are *disabled* for batches containing rare class samples to avoid diluting their scarce training signal.

5.7 Evaluation Protocol

- Leakage-free split:** 90%/10% stratified train/val split on original images only (seed 42). Augmented copies follow their parent’s split. Validation contains originals only (27,081 train / 2,891 val).
- Macro-F1** as primary metric (same as Kaggle leaderboard).
- Test inference:** CLAHE-preprocessed crops, one base pass plus 8 stochastic TTA rounds (random H/V flips and 90° rotations), with logits averaged.

Table 5: Final hyperparameters for `deep_9 no-morph`.

Parameter	Value
Backbone	ConvNeXt-Base (torchvision, ImageNet)
Attention	SE-style channel gate (1024→128→1024, sigmoid)
Head	MLP 1024→512→256→13, ReLU, dropout 0.3
Parameters	88.49M total; 0.92M trainable in phase 1
Preprocessing	Offline HSV crop (pad 20) + CLAHE on HSV-V (clip 2.0, 8×8), train/val/test
Split	90/10 stratified on original images (seed 42); 27,081 train / 2,891 val
Sampler	WeightedRandomSampler, weights $\propto n_c^{-1/2}$
Optimizer	AdamW [19]
Phase-1 LR (gate + head)	3×10^{-4} , cosine warm restarts, 5 epochs
Phase-2 LR (all)	1×10^{-4} (backbone and head, same LR)
Weight decay	1×10^{-4}
Batch size	16×4 accumulation steps (effective 64)
Epoch budget	60 max; best epoch 21, early stop at 31
Early stopping	Patience 10 (val macro-F1)
Loss	Focal Loss ($\gamma = 2.0$) + label smoothing ($\epsilon = 0.05$)
Batch mixing	CutMix ($\alpha = 1.0, p = 0.5$) / Mixup ($\alpha = 0.2$); off for batches with rare classes
Precision	AMP fp16
Gradient clipping	Norm 1.0
Input resolution	224×224
TTA	Base pass + 8 stochastic rounds (H/V flip, rot90), logits averaged
Hardware	$1 \times$ RTX 3090; ≈ 84 s / ≈ 140 s per epoch (frozen / fine-tune)

6 Iterative Development: The Full Journey

A key aspect of this work was iterative refinement across 9 experiments. Each iteration addressed specific failure modes identified in the previous one. Table 6 summarizes all experiments, and the following subsections detail each.

6.1 `deep_1`: EfficientNet-B3 Baseline

The first deep learning attempt used EfficientNet-B3 [22] (12M parameters, ImageNet pretrained) with a simple classifier head (Dropout(0.4) + Linear), torchvision transforms (basic flips, rotation, color jitter), WeightedRandomSampler, and weighted CrossEntropyLoss, originally run with 5-fold stratified K-Fold cross-validation and three TTA variants (original, H-flip, V-flip) at inference.

Note: the original 5-fold run was saved without outputs, so its score cannot be checked. The same model, loss and sampler were therefore re-run under a shorter protocol (a single stratified 85/15 split, at most 15 epochs instead of up to 30 per fold $\times$ 5 folds); the numbers above come from this re-run (`results/deep_1_rerun/`).

Results (re-run): val macro-F1 = 0.552, accuracy = 0.716. Per-class F1: BA=0.92, BL=0.83, LY=0.82, SNE=0.81, MO=0.79, EO=0.78, MY=0.52, PMY=0.40, PC=0.34, MMY=0.30, VLY=0.27, PLY=0.25, BNE=0.14. This score is noisy: the validation macro-F1 still oscillated

between 0.44 and 0.55 over the last epochs, and an earlier unseeded run of the same script reached 0.489. Even under-trained, transfer learning is already on par with the classical pipeline (0.509, under a different protocol), with rare classes remaining the main weakness – a pattern that persists across every later iteration.

6.2 `deep_2`: CBAM Attention + Nucleus Cropping

Motivation: Improve focus on the nucleus (the primary discriminative feature) and add domain-specific augmentation.

Changes: Switched to the `timm` library for feature extraction with Spatial CBAM [9] (channel + spatial attention) on the last feature map. Added HSV-based nucleus cropping at runtime and a comprehensive Albumentations [17] pipeline including staining variation (HSV shifts, RGBShift, CLAHE). Introduced label smoothing [16] (0.1), Mixup ($\alpha=0.4$), gradient clipping, and CosineAnnealing-WarmRestarts. Tested CNN+RF and CNN+SVM hybrids on extracted backbone features. Used a single 85/15 split for faster iteration.

Results: DL-only macro-F1 = 0.552, but CNN+SVM achieved **0.633**—a significant finding that SVM on CNN features substantially outperformed end-to-end training with the small EfficientNet backbone. Training was slow because HSV cropping was still performed at runtime. Per-class: SNE=0.96, LY=0.94, but PLY=0.00, PMY=0.22, BNE=0.31.

Key insight: For smaller backbones, the learned representation benefits from a classical classifier (SVM/RF) rather than the trained softmax head.

6.3 `deep_3`: Late Fusion + Wiener/Laplacian + 4th Channel

Motivation: Inject explicit morphological knowledge that the CNN might miss, inspired by classical segmentation literature [2].

Changes: Two-branch Late Fusion model: (1) EfficientNet-B3 + CBAM with 4-channel input (RGB + nucleus mask as 4th channel), (2) MLP on 8 handcrafted morphometry features (roundness, solidity, compactness, eccentricity, C/N ratio, normalized area, perimeter, extent). Added Wiener filter denoising and Laplacian edge enhancement as preprocessing. Mixup applied to both branches with consistent λ /permutation.

Results: DL-only F1 = 0.552, Fused+SVM = **0.641**—a marginal +0.8pp over `deep_2`. The additional complexity (4th channel, Wiener filtering) did not translate into meaningful gains for the DL head. Training became considerably slower because of the Wiener filtering, while macro-F1 was nearly unchanged.

Key insight: Adding complexity to the input (4th channel, denoising filters) increases computational cost without proportional performance gains.

Table 6: Summary of all deep learning iterations. F1 = validation macro-F1. “Best approach” indicates whether end-to-end DL or a hybrid (backbone features fed to RF/SVM) performed better.

Iter.	Backbone	Key Innovation	Best F1	Best Approach	Key Finding
deep_1	EfficientNet-B3	Baseline, weighted CE	0.552 [†]	DL	Established initial baseline
deep_2	EffNet-B3 + CBAM	HSV crop, Albumentations, hybrid	0.633	CNN+SVM	SVM on CNN features > DL
deep_3	EffNet-B3 + CBAM	Late fusion, Wiener/Laplacian, RGBM	0.641	Fused+SVM	Marginal gain from fusion
deep_4	EffNet-B3 + CBAM	Gautam 2014 segmentation, 4 feats	0.591	Fused+RF	Rigid pipeline <i>hurt</i> perf.
deep_5	ConvNeXt-Base	Focal Loss, CutMix, 2-phase, offline	0.822	DL	Inflated by data leakage!
deep_6	ConvNeXt-Base	Leakage fix, visual QC	0.700	DL	True performance after leakage fix
deep_7	ResNet50 + SpatialAtt	Late fusion, 4 morpho feats	0.651	DL	Weaker backbone $\Rightarrow$ lower F1
deep_8	ResNeXt101 + CBAM	Multi-task aux heads, 6 morpho feats	0.665	DL	Complex but no gain over deep_6
deep_9	ConvNeXt + SE	SE gate, HSV-V CLAHE, with-/no-morph ablation	0.705	DL	Deployed model ; morpho feats $\approx$ no effect

[†]deep_1’s original run (5-fold CV, up to 30 epochs/fold) has no saved outputs in the source notebook and could not be verified; 0.552 is from a seeded re-run (`scripts/deep_1_rerun.py`) under a reduced protocol (single stratified 85/15 split, 15-epoch cap) kept within a practical compute budget – see §6.1. It is not directly comparable to the other rows’ full training budgets.

6.4 deep_4: Literature Segmentation Pipeline (Failed Experiment)

Motivation: Replace our ad-hoc nucleus segmentation with the published pipeline of Gautam and Bhadauria [3] (contrast stretch + histogram equalization + add/subtract isolation + minimum filter + Otsu + morphological closing + border clearing).

Changes: Adopted the paper’s exact segmentation pipeline. Reduced handcrafted features from 8 to 4 (area, perimeter, eccentricity, circularity). Removed class weights from the loss function (only WeightedRandomSampler) to fix NaN gradient explosions caused by double imbalance correction. Added NaN guards (`torch.nan_to_num` on inputs, skip NaN-loss batches).

Results: Performance *dropped* to F1 = **0.591**. Per-class: SNE=0.95, LY=0.90, but BNE=0.24, PMY=0.27, PC=0.33—a clear regression in macro-F1 from deep_3 (0.641 $\rightarrow$ 0.591), driven by the rare classes.

Key insight: Classical segmentation pipelines designed for clean laboratory conditions do not generalize to heterogeneous real-world staining. This failure strongly motivated abandoning complex feature engineering in favor of end-to-end learning.

6.5 deep_5: The ConvNeXt Pivot + Data Leakage Discovery

Motivation: After diminishing returns from complex feature engineering (deep_2–deep_4, all in the 0.55–0.64 range), try a fundamentally stronger backbone with a simpler pipeline.

Changes: Complete pipeline redesign—removed all attention modules, handcrafted features, and mask channels. ConvNeXt-Base [18] (88M params, $\sim 7\times$ larger than EfficientNet-B3), 3-layer MLP head, Focal Loss [15] ($\gamma=2.0$), CutMix [20]+Mixup [21] (disabled for rare classes), 2-phase training (warmup then finetune), AMP mixed precision, gradient accumulation, offline oversampling of rare classes to

median count, offline HSV pre-cropping (removing the runtime cropping bottleneck), stochastic TTA (8 rounds).

Results: Achieved validation F1 = **0.822**—a massive jump from the best previous result (0.641). *However, this was inflated by data leakage.*

The leakage: Augmented copies of images were split *after* augmentation, meaning an augmented copy of image X could appear in validation while the original X was in training. The model effectively memorized these images.

6.6 deep_6: Leakage Fix — True Performance

Motivation: Eliminate data leakage and measure true generalization.

Changes: Split on *original* image identity before assigning augmented copies. Validation set contains only original images (no augmented copies). Added comprehensive visual QC: raw vs. cropped comparison per class, crop size distributions, failure detection (crops covering >90% of original area), train vs. test crop ratio comparison.

Results: True macro-F1 = 0.6994, accuracy = 0.936. The 12.3-point absolute drop from deep_5 (0.822 $\rightarrow$ 0.699, $\approx 15\%$ relative) demonstrates the severity of the leakage. Training reached epoch 21 (early stopping, patience=5), best at epoch 16. A hybrid approach (backbone features $\rightarrow$ Random Forest) was tested but underperformed (F1 = 0.68).

This establishes the leakage-safe architecture and evaluation protocol that all later iterations (deep_7–deep_9) build on. It is not, however, the best-scoring iteration: deep_9 no-morph, which adds one lightweight attention gate and HSV-V CLAHE on top of this architecture, scores slightly higher (0.7052, §6.9) and is the model actually deployed.

6.7 deep_7: ResNet50 + Spatial Attention + Late Fusion

Motivation: Test whether injecting explicit morphological features via late fusion can improve upon the pure ConvNeXt

pipeline. Use ResNet50 [23] as an alternative backbone.

Changes: ResNet50 (24.8M params) with a custom Spatial Attention Gate (CBAM-style: channel avg+max pooling $\rightarrow$ 7×7 conv $\rightarrow$ sigmoid). Added a tabular branch with 4 handcrafted nucleus features (area, perimeter, eccentricity, circularity) extracted via Otsu + morphological operations. Image embedding (256-dim) concatenated with tabular embedding (64-dim) before the classifier ($320 \rightarrow 256 \rightarrow 13$).

Results: 5-fold CV mean F1 = 0.646 (± 0.018), holdout F1 = 0.651. Ablation showed morpho features added +0.017 F1 over image-only (0.633 $\rightarrow$ 0.651). The hybrid Features+RF scored 0.593 (worse than end-to-end). Per-class: BNE=0.49, PMY=0.32, PLY=0.00 remained critical.

Key insight: ResNet50 (24.8M params) is significantly weaker than ConvNeXt-Base (88M params) for this task. The morphological features provide a small but inconsistent benefit with this backbone.

6.8 deep_8: ResNeXt101 + CBAM + Multi-Task

Motivation: Test a different backbone family of similar size to ConvNeXt-Base, combined with attention and multi-task supervision to regularize learning.

Changes: ResNeXt101-32x8d [24] (~ 87 M params) with full CBAM attention [9] (channel + spatial). Late fusion with 6 morphometric features (nucleus/cell area ratio, circularity, lobe count, cytoplasm saturation, eccentricity, log area). Two auxiliary heads: (1) regression for 3 morphometry pseudo-labels (nucleus/cell ratio, circularity, cytoplasm saturation), (2) 5-class nuclear lobe count prediction. Confusion-aware margin penalty for known confusing pairs (MO/EO, SNE/EO, BNE/EO).

Results: 5-fold CV mean F1 = 0.680 (± 0.022), holdout F1 = 0.665. A zero-out ablation—morphological inputs set to zero *at inference only*, on the model trained with them (no retraining)—dropped F1 from 0.665 to 0.492 (-0.173). This measures how much the trained network *relies* on its morphology branch, not the gain that adding the features would bring over a model trained without them. However, the overall F1 (0.665) still did not exceed deep_6 (0.700). Per-class: BA=0.96, EO=0.96, but BNE=0.42, PMY=0.38, PLY=0.00.

Key insight: With a backbone of essentially the same size as ConvNeXt-Base (~ 87 M vs. ~ 88 M params), plus CBAM attention and multi-task supervision, the result still does not beat the simpler ConvNeXt-Base + MLP. Parameter count alone does not explain the gap; the backbone family/architecture mattered more than the added modules.

6.9 deep_9: ConvNeXt-Base + SE Attention (with-morph vs. no-morph)

Motivation: Definitive controlled experiment—does adding handcrafted morphological features help when using ConvNeXt-Base?

Changes: ConvNeXt-Base with a lightweight SE-style channel attention gate [10] on the 1024-dim features, CLAHE on the HSV V channel as deterministic preprocessing, and the augmentation of Table 4. Two otherwise identical experiments: deep_9 with-morph (6 morpho features fused via an MLP branch) and deep_9 no-morph (pure deep features).

Results:

- deep_9 with-morph: val macro-F1 = 0.7036, accuracy = 0.9381
- deep_9 no-morph: val macro-F1 = **0.7052**, accuracy = 0.9367 — the best validated macro-F1 across all 9 iterations, and the model deployed (checkpoint at models/best_model.pt, §5.1)

The morphological features make *essentially no difference* (0.7036 vs. 0.7052, a 0.0016 macro-F1 difference, too small to be meaningful on a single validation split). Both are comparable to deep_6 (0.6994), indicating that ConvNeXt-Base already captures the morphological information that these handcrafted features provide. A Features+RF hybrid on the gated deep_9 no-morph features reached 0.6801, again below end-to-end training. The dominant confusions remain the same: for deep_9 no-morph, BNE $\rightarrow$ SNE accounts for 41.0% of BNE and VLY $\rightarrow$ LY for 24.3% of VLY.

Key insight: This is the definitive answer to “do handcrafted features help?”—*not with this backbone*. ConvNeXt-Base learns representations that subsume explicit morphometry. With the other backbones (EfficientNet-B3, ResNet50, ResNeXt101) the fused models did make use of the morphology branch (deep_7 +0.017; deep_8 relies on it, see above), but none of them reached ConvNeXt’s level. Since deep_9 no-morph scores marginally higher and is simpler (no unused morphology branch), it is the deployed configuration.

7 Results and Discussion

7.1 Approach Comparison

Table 7: Comparison of approaches (validation macro-F1).

Approach	Macro-F1	Accuracy
Classical ML (best: LightGBM)	0.509	0.832
DL baseline (EfficientNet-B3) [†]	0.552	0.716
DL deployed (ConvNeXt-Base, deep_9 no-morph)	0.705	0.937

[†]Reduced protocol (single 85/15 split, 15-epoch cap) – see §6.1, not directly comparable to the other two rows’ full training budgets.

The deep learning approach with ConvNeXt-Base (0.705 macro-F1) outperforms both classical ML (0.509) and the DL baseline with EfficientNet-B3 (0.552). The gap between accuracy (0.937) and macro-F1 (0.705) reflects the extreme class imbalance: the model correctly classifies common classes (SNE, LY) but struggles with rare ones.

7.2 Per-Class Analysis (deployed model, deep_9 no-morph)

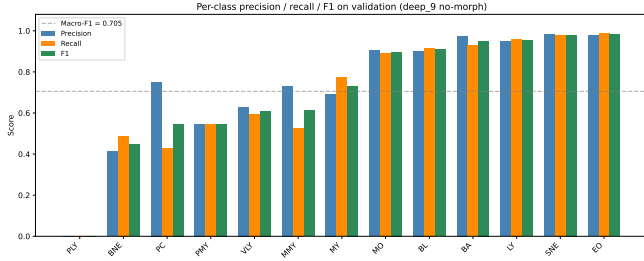


Figure 7: Per-class precision, recall and F1 of the deployed model (deep_9 no-morph, validation set). Dashed line = macro-F1 = 0.705.

From the deployed model’s evaluation (2,891 validation images), we observe three tiers of classification difficulty:

Well-classified ($F1 \geq 0.89$): EO (0.983), SNE (0.981), LY (0.953), BA (0.951), BL (0.909), MO (0.897). These have abundant data or highly distinctive morphology (BA with dark granules, EO with bilobed nucleus).

Moderate ($F1 0.54\text{--}0.75$): MY (0.731), MMY (0.613), VLY (0.611), PC (0.545), PMY (0.545). Limited training data combined with morphological overlap.

Critical ($F1 < 0.50$): BNE (0.447), PLY (0.000). BNE vs. SNE differs only in nuclear segmentation degree. PLY has only 11 training images (1 in validation), making reliable classification impossible.

7.3 Confusion Analysis

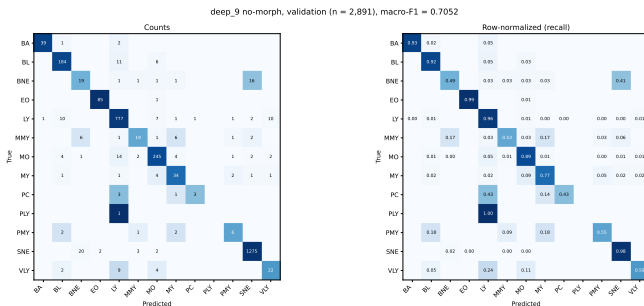


Figure 8: Confusion matrix (counts and normalized recall) from the deployed model (deep_9 no-morph).

The confusion matrix reveals biologically meaningful patterns:

1. **BNE ↔ SNE** (SNE→BNE 20, BNE→SNE 16, i.e. 41.0% of BNE): Band and segmented neutrophils share the same lineage, differing only in nuclear segmentation degree—a distinction that challenges even expert hematologists.

2. **Lymphoid/mononuclear cells** (MO→LY 14, BL→LY 11, LY→VLY 10, LY→BL 10, VLY→LY 9, i.e. 24.3% of VLY): lymphocytes, variant lymphocytes, blasts and monocytes are all mononuclear cells with overlapping size and chromatin appearance.

3. **Granulocyte maturation chain** (MMY→MY 6, MMY→BNE 6): successive maturation stages with gradually evolving nuclear shape (round → indented → bean-shaped → band).

These confusions reflect genuine biological continua, not model failures.

7.4 Qualitative Error Analysis

To complement the confusion matrix, we inspected representative misclassified validation images. Figure 9 shows the four errors made with the highest confidence; three of them confuse neighbouring stages or subtypes of the same lineage (band vs. segmented neutrophil, blast vs. lymphocyte, pro-lymphocyte vs. lymphocyte).

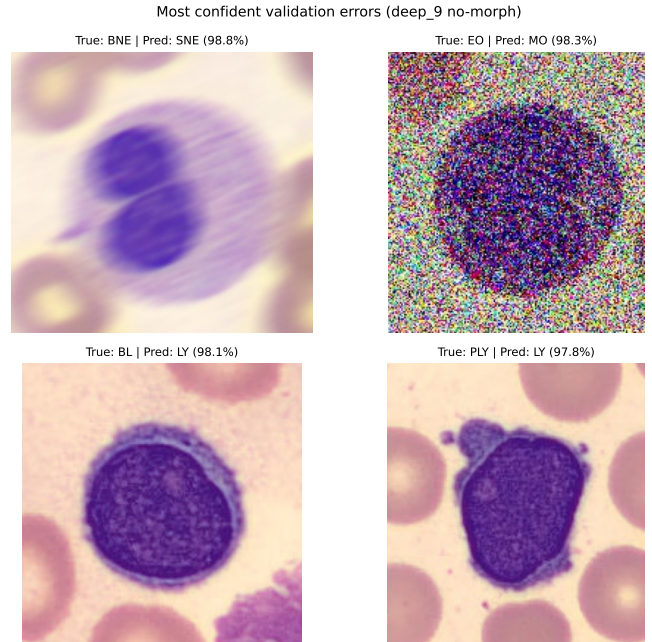


Figure 9: The four most confident validation errors of the deployed model (deep_9 no-morph): BNE predicted as SNE, EO as MO, BL as LY and PLY as LY, each with a confidence above 97%.

Across these examples, errors are largely explained by borderline morphology (e.g., partially segmented nucleus, intermediate chromatin condensation, weak granulation) rather than obvious model artifacts. This supports the interpretation that a substantial fraction of residual error reflects class overlap in the biological continuum.

7.5 Why Deep Learning Outperforms Classical ML

The performance gap (0.196 F1, 0.705 vs. 0.509) arises from three factors:

1. **Segmentation robustness:** The DL pipeline only needs a coarse HSV crop, which tolerates imprecise boundaries, while classical ML requires precise nucleus/cytoplasm boundaries; every segmentation failure corrupts the entire feature vector.
2. **Feature learning:** ConvNeXt learns hierarchical features capturing subtle chromatin patterns, granule distributions, and nuclear texture at multiple scales—information that GLCM or LBP can only partially encode.
3. **Transfer learning:** ImageNet pre-training provides general visual representations (edges, textures, shapes) that transfer well to microscopy, compensating for the limited medical training data.

7.6 Lessons from the Iterative Journey

The 9-iteration journey yielded several generalizable insights:

1. **The backbone family matters more than added modules:** The single largest performance jump came from switching EfficientNet-B3 (12M) to ConvNeXt-Base (88M). Size alone is not the explanation: ResNeXt101 (~87M, deep_8) with CBAM, fusion and multi-task heads reached only 0.665. No amount of attention, fusion, or multi-task engineering compensated for a less suitable backbone.
2. **Handcrafted features matter for weaker pipelines, not for ConvNeXt:** Late fusion gave small gains with EfficientNet-B3 (deep_3 vs. deep_2, +0.008) and ResNet50 (deep_7, +0.017), and the ResNeXt101 model strongly relied on its morphology branch (zero-out ablation -0.173 at inference), but it provided no benefit for ConvNeXt-Base (deep_9 with-morph vs. no-morph: 0.0016 difference).
3. **Data leakage is insidious:** The deep_5 $\rightarrow$ deep_6 episode (0.82 $\rightarrow$ 0.70) demonstrates how augmentation+CV can create subtle leakage. Always split by original image identity.
4. **Simplicity wins, mostly:** The deployed model (deep_9 no-morph) is deep_6's backbone + MLP plus one lightweight SE gate and CLAHE preprocessing – the only additions that helped at all, and only marginally (+0.006 F1, 0.7052 vs. 0.6994). Heavier designs (CBAM, spatial attention, multi-task, late fusion) added engineering complexity without consistent gains.

5. **Hybrid classifiers suit small models:** CNN+SVM/RF outperformed end-to-end training for EfficientNet-B3 (deep_2/3) but not for ConvNeXt-Base, suggesting that larger backbones produce better-separated feature spaces that don't need kernel methods.

7.7 Computational Cost and Clinical Feasibility

Although ConvNeXt-Base is a large model (~88M parameters), the offline HSV cropping stage substantially reduces irrelevant background and accelerates training/inference. In our implementation, moving from online to offline cropping removed the main preprocessing bottleneck (deep_5 onwards); the deployed model trains at ≈ 84 s/epoch with a frozen backbone and ≈ 140 s/epoch during fine-tuning on a single RTX 3090.

At inference, the deployed pipeline performs a single forward pass per crop (plus optional TTA for leaderboard scoring). This makes the non-TTA mode suitable for near-real-time triage scenarios, while TTA can be reserved for high-confidence offline batch reporting.

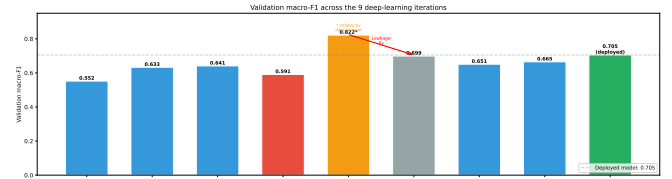


Figure 10: Evolution of validation macro-F1 across iterations. The deep_5 $\rightarrow$ deep_6 drop reflects leakage correction, not regression.

8 Conclusion and Future Work

We explored both classical ML and deep learning for 13-class WBC classification through an extensive iterative process spanning 9 deep learning experiments. The classical approach, while providing interpretable features, is fundamentally limited by segmentation failures and the expressiveness gap between handcrafted and learned features.

The deep learning journey revealed that the most effective design is close to the simplest one tried. After exploring CBAM attention (deep_2), late fusion with handcrafted morphometry (deep_3, 7, 8, 9), 4-channel RGBM input (deep_3), Wiener/Laplacian preprocessing (deep_3), paper-based segmentation (deep_4), spatial attention (deep_7), multi-task auxiliary heads (deep_8), and confusion-aware margin penalties (deep_8), the deployed model (deep_9 no-morph) is:

1. **ConvNeXt-Base backbone** with ImageNet transfer learning (88.5M parameters in total).

2. **Lightweight SE-style attention gate** on pooled features (263K params) – together with CLAHE, the only addition that gave a (marginal) gain.
3. **3-layer MLP head** ($1024 \rightarrow 512 \rightarrow 256 \rightarrow 13$).
4. **Offline HSV cropping** for robust background removal, followed by **CLAHE on the HSV V channel** at train and test time.
5. **Focal Loss + CutMix/Mixup** for class imbalance, with rare-class protection.
6. **2-phase training** (5-epoch frozen warmup, then full fine-tuning).
7. **Leakage-free evaluation** with original-identity-based splits.

This achieves macro-F1 = 0.705 on validation, outperforming classical ML (0.509) and the initial DL baseline (0.552). The iterative journey demonstrated that *the choice of backbone family and training robustness matter more than architectural complexity* – attention and fusion mostly did not help, with one narrow exception.

Future directions include: (1) higher input resolution (384×384), (2) stain-specific augmentation (HSV shifts, RGBShift), which the deployed model does not use, (3) multi-backbone ensemble (ConvNeXt + EfficientNetV2 + Swin Transformer), (4) hierarchical classification exploiting the granulocyte maturation chain (PMY→MY→MMY→BNE→SNE), (5) Macenko stain normalization [25] as preprocessing, and (6) external data or synthetic generation (GANs/diffusion) for the PLY class (only 11 images).

References

- [1] Khamael Al-Dulaimi, Jasmine Banks, Vinod Chandran, Inmaculada Tomeo-Reyes, and Kien Nguyen Thanh. Classification of white blood cell types from microscope images: Techniques and challenges. In A. Méndez-Vilas and E. Torres-Hergueta, editors, *Microscopy Science: Last Approaches on Educational Programs and Applied Research*, number 8 in Microscopy Book Series, pages 17–25. Formatex Research Center, Spain, 2018.
- [2] Seyed Hamid Rezatofghi and Hamid Soltanian-Zadeh. Automatic recognition of five types of white blood cells in peripheral blood. *Computerized Medical Imaging and Graphics*, 35(4):333–343, 2011.
- [3] Anjali Gautam and Harvindra Bhadauria. Classification of white blood cells based on morphological features. In *International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 2363–2368, 2014.
- [4] Thanh Tran, Oh-Heum Kwon, Ki-Ryong Kwon, Suk-Hwan Lee, and Kyung-Won Kang. Blood cell images segmentation using deep learning semantic segmentation. In *IEEE International Conference on Electronics and Communication Engineering (ICECE)*, 2018.
- [5] Feiwei Qin, Nannan Gao, Yong Peng, Zizhao Wu, Shuying Shen, and Artem Grudtsin. Fine-grained leukocyte classification with deep residual learning for microscopic images. *Computer Methods and Programs in Biomedicine*, 162:243–252, 2018.
- [6] Madhumala Ghosh, Devkumar Das, Subhodip Mandal, Chandan Chakraborty, Mallika Pal, Ashok K. Maity, Surjya K. Pal, and Ajoy K. Ray. Statistical pattern analysis of white blood cell nuclei morphometry. In *Proceedings of the IEEE Students’ Technology Symposium*, IIT Kharagpur, India, 2010.
- [7] Furqan Rustam, Naila Aslam, Isabel De La Torre Díez, Yaser Daanial Khan, Juan Luis Vidal Mazón, Carmen Lili Rodríguez, and Imran Ashraf. White blood cell classification using texture and RGB features of over-sampled microscopic images. *Healthcare*, 10(11):2230, 2022.
- [8] Mesut Toğaçar, Burhan Ergen, and Zafer Cömert. Classification of white blood cells using deep features obtained from convolutional neural network models based on the combination of feature selection methods. *Applied Soft Computing*, 97:106810, 2020.
- [9] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. CBAM: Convolutional block attention module. In *European Conference on Computer Vision (ECCV)*, pages 3–19, 2018.
- [10] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7132–7141, 2018.
- [11] Erik Reinhard, Michael Ashikhmin, Bruce Gooch, and Peter Shirley. Color transfer between images. *IEEE Computer Graphics and Applications*, 21(5):34–41, 2001.
- [12] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [13] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 3146–3154, 2017.
- [14] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In

ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pages 2623–2631, 2019.

- [15] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.
- [16] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2016.
- [17] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.
- [18] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11976–11986, 2022.
- [19] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [20] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. CutMix: Regularization strategy to train strong classifiers with localizable features. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6023–6032, 2019.
- [21] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.
- [22] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning (ICML)*, pages 6105–6114, 2019.
- [23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [24] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1492–1500, 2017.
- [25] Marc Macenko, Marc Niethammer, J.S. Marron, David Borland, John T. Woosley, Xiaojun Guan, Charles

Schmitt, and Nancy E. Thomas. A method for normalizing histology slides for quantitative analysis. In *IEEE International Symposium on Biomedical Imaging (ISBI)*, pages 1107–1110, 2009.